데이터 분석을 위한 NumPy

학습목표

1) 라이브러리를 import 로 불러온다

표준·외부 라이브러리, 별칭 np·pd

2) NumPy 배열로 여러 값을 한 번에 계산한다

벡터화·브로드캐스팅, 인덱싱, 조건 선택, 배열 메서드(mean·sum 등), np.where

3) 2차원 배열의 구조와 방향을 이해한다

shape, axis, reshape·.T

DAY 3의 흐름

노트북 0~1절

import 와 NumPy 의 역할

별칭, 리스트 vs 배열, 벡터화

노트북 2절 1)~8)

NumPy 기초

구조, 만들기, 난수, 인덱싱, 조건, 연산, nan, 통계, where

노트북 2절 9)~12)

2차원 배열

shape, axis, reshape, .T

라이브러리와 import

라이브러리와 import

용어	의미	예
모듈	함수·클래스·변수를 담은 파일 하나	math, datetime
패키지	관련된 모듈 여러 개를 묶은 것	numpy, pandas
라이브러리	특정 목적의 모듈과 패키지를 부르는 넓은 말	NumPy, pandas, Matplotlib

표준 라이브러리와 외부 라이브러리

표준 라이브러리(math, os, datetime)는 Python을 설치하면 함께 들어 있다.
외부 라이브러리(NumPy, pandas)는 따로 설치해야 하는데, Colab에는 미리 설치되어 있어 import만 하면 된다.
내 컴퓨터에서 쓰려면 pip install pandas 로 설치한다.

별칭 np, pd 는 전 세계 개발자가 따르는 관례(convention)다

Python 모듈(한국어): https://docs.python.org/ko/3.13/tutorial/modules.html

모듈 · 패키지 · 라이브러리

모듈은 코드와 기능을 담은 단위이고, 패키지는 관련 모듈을 조직하는 구조이다.
라이브러리는 재사용 가능한 기능을 제공하는 모듈과 패키지 등을 통틀어 부르는 말이다.



모듈

기능을 담은 파일 하나

math (파일 하나)

→



패키지

모듈 여러 개를 묶은 폴더

numpy (폴더 안에 모듈 여러 개)

→



라이브러리

특정 목적의 도구 모음

NumPy, pandas, Matplotlib

설치: pip install pandas

불러오기: import pandas as pd

Python 모듈·패키지(한국어): https://docs.python.org/ko/3.13/tutorial/modules.html#packages · Python 모듈 설치(한국어): https://docs.python.org/ko/3.13/installing/index.html

라이브러리와 import

```
import math                 # 표준 라이브러리: 설치 없이 바로
print(math.sqrt(16))        # 4.0

import numpy as np          # 별칭(alias): 긴 이름 대신 쓰는 짧은 이름. np 가 관례
import pandas as pd         # 별칭 pd

from math import sqrt       # 필요한 이름만 가져오기
print(sqrt(16))             # math. 없이 바로 쓴다
```

`import 모듈`

모듈.함수() 처럼 모듈 이름을 붙여 쓴다

`from 모듈 import 함수`

함수() 처럼 이름만 바로 쓴다

NumPy 배열 단위 연산과 브로드캐스팅

NumPy 배열 단위 연산

Python 리스트

```
python_sales = [140, 180, 95, 230, 160]

# 각 값에 1000을 곱하려면 반복문이 필요하다
result = []
for value in python_sales:
    result.append(value * 1000)
```

값 하나씩 처리한다. 데이터가 수십만 건이면 느리고 코드가 길다.

NumPy 배열

```
import numpy as np
sales = np.array([140, 180, 95, 230, 160])

# 배열 전체에 한 번에 적용된다
result = sales * 1000
print(sales.mean(), sales.max())
# 161.0 230
```

배열 전체를 한 번에 계산한다. 반복문 없이 짧고 빠르다. pandas가 내부에서 NumPy를 쓴다.

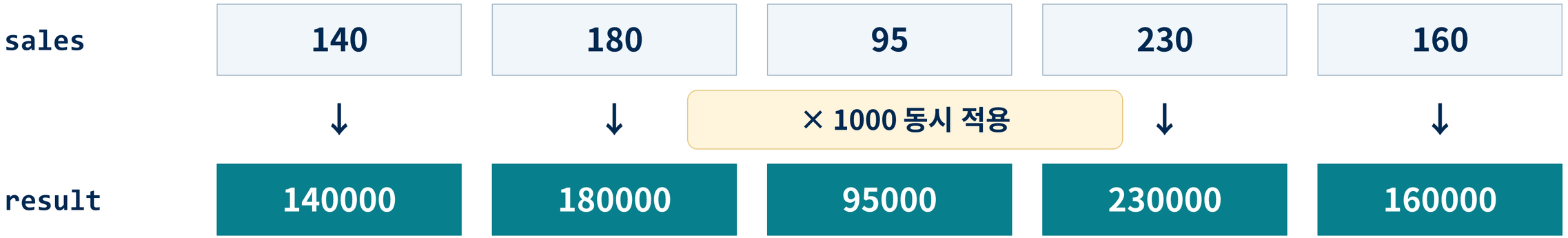
NumPy 기초: https://numpy.org/doc/stable/user/absolute_beginners.html

NumPy의 역할: 배열 단위 연산

NumPy는 같은 자료형의 값을 배열 단위로 묶어 한 번에 연산(벡터화)하는 수치 계산 라이브러리다.

벡터화(vectorization): 반복문을 명시적으로 쓰지 않고 배열 단위로 연산을 적용하는 것.

```
result = sales * 1000
```



리스트와 배열의 곱셈

1) 리스트: 목록을 반복

```
sales = [60, 90]
print(sales * 2)
# [60, 90, 60, 90]
```

2) 배열: 각 값에 곱하기

```
import numpy as np
sales = np.array([60, 90])
print(sales * 2)
# [120 180]
```

브로드캐스팅

배열(값 5개) + 숫자 하나가 계산되는 이유

sales

[140 180 95 230 160]

배열 5개

→

+ 10

→ **[10 10 10 10 10]**

숫자 하나를 5개로 늘린다

→

[150 190 105 240 170]

같은 위치끼리 더한다

```
sales + 10              # 브로드캐스팅
sales * 1.1             # 10% 인상도 같은 원리
sales - sales.mean()  # 평균(숫자 하나)을 모든 값에서 뺀다
```

브로드캐스팅(broadcasting)이란

NumPy에서 모양(shape)이 다른 배열이나 스칼라(숫자)를 연산할 때, 호환되는 차원의 크기를 맞춰 연산을 수행하는 기능이다.

브로드캐스팅: https://numpy.org/doc/stable/user/basics.broadcasting.html

브로드캐스팅

배열의 shape를 오른쪽부터 비교하여 대응하는 차원의 크기가 같거나 한쪽이 1이면 연산할 수 있다. 조건을 만족하지 않으면 오류가 발생한다.

✅ | 140 | 180 | 95 | 230 | 160 | + | 10

숫자 하나는 5개로 늘어난다

결과: [150 190 105 240 170]

✅ | 140 | 180 | 95 | 230 | 160 | + | 10 | `np.array([10])`

값 1개짜리 배열도 숫자 하나처럼 5개로 늘어난다

결과: [150 190 105 240 170]

✅ | 140 | 180 | 95 | 230 | 160 | + | 2 | 4 | 6 | 8 | 10

개수가 같으면 같은 위치끼리 계산한다

결과: [142 184 101 238 170]

❌ | 140 | 180 | 95 | 230 | 160 | + | 2 | 4 | 6

5개와 3개는 늘려서 맞출 방법이 없다

결과: 오류(ValueError)

```
ValueError: operands could not be broadcast together with shapes (5,) (3,)
```

NumPy 기초 · 배열 만들기 · 인덱싱 · 조건 · 결측

배열

배열(array)은 같은 자료형의 값을 순서대로 담은 자료구조다. 자료형이 섞이면 하나로 통일된다

구분	리스트	배열
요소의 자료형	서로 다른 자료형을 섞을 수 있다	하나의 자료형으로 통일된다
연산 방식	반복문으로 하나씩 처리한다	전체에 일괄 적용한다 (벡터화)
생성	[140, 180, 95]	np.array([140, 180, 95])
인덱싱·슬라이싱	sales[0], sales[1:3]	동일하다

배열 생성: https://numpy.org/doc/stable/user/basics.creation.html · np.arange(): https://numpy.org/doc/stable/reference/generated/numpy.arange.html

배열

리스트

자료형 혼합 가능

인덱스	값	자료형
0	7	int
1	"부산"	str
2	4.8	float

배열

단일 자료형으로 통일

인덱스	값	자료형
0	140	int64
1	180	int64
2	95	int64

배열의 인덱스

인덱싱: 위치 번호로 값 하나를 꺼낸다
슬라이싱: [시작:끝:간격]
시작:끝 범위로 여러 개를 꺼낸다(끝 미포함) · 규칙은 리스트와 같다



sales[0] → 140

sales[1:3] → [180 95]

sales[::-1] → [95 180 140]

sales[::-1]: 처음부터 끝까지 거꾸로 한 칸씩 = 뒤집기 · 리스트·문자열도 같다 ("abc"[::-1] → "cba")

NumPy 인덱싱: https://numpy.org/doc/stable/user/basics.indexing.html

배열을 만드는 주요 방법

함수	의미	예시
np.array(리스트)	리스트를 배열로 변환	np.array([140, 180, 95])
np.arange(시작, 끝)	시작부터 끝 앞까지 연속된 숫자	np.arange(2, 8) → 2~7
np.zeros(n)	0이 n개인 배열	np.zeros(4) → [0. 0. 0. 0.]
np.ones(n)	1이 n개인 배열	np.ones(4) → [1. 1. 1. 1.]
np.linspace(시작, 끝, 개수)	구간을 균등하게 나눈 숫자	np.linspace(0, 10, 5)→ [0. 2.5 5. 7.5 10.]

난수란 (random number)

난수는 무작위로 선택된 것처럼 나타나는 수이다.

=> 주사위 눈처럼 다음에 무엇이 나올지 예측하기 어려운 숫자이다. 컴퓨터는 정해진 계산만 하기 때문에, 계산 규칙으로 무작위처럼 보이는 숫자를 만든다. 이를 의사 난수(Pseudo-Random Number) 라고 한다.

같은 코드를 두 번 실행하면

```
np.random.randint(50, 100, size=5)
```

1번째 실행 **[97 54 75 53 69]**

2번째 실행 **[73 89 78 64 73]**

실행 결과 예 (매번 다름)
50 이상 100 미만에서 5개를 뽑는다

NumPy 난수(Random sampling): https://numpy.org/doc/stable/reference/random/index.html · randint: https://numpy.org/doc/stable/reference/random/generated/numpy.random.randint.html

난수와 seed

seed(시드)는 난수를 만드는 계산의 시작값이다.
시작값이 같으면 그 뒤에 나오는 난수의 순서가 똑같다.

seed 를 정하고 두 번 실행하면

```
np.random.seed(0)
np.random.randint(50, 100, size=5)
```

1번째 실행

```
[94 97 50 53 53]
```

2번째 실행

```
[94 97 50 53 53]
```

알아 둘 것

1) 컴퓨터의 난수는 정해진 계산으로 만든 '난수처럼 보이는 수'(의사 난수)다. 그래서 시작값을 알면 같은 순서로 다시 만들 수 있다

2) seed 숫자 자체(0, 42)에는 뜻이 없다. 어떤 수를 쓰든 '고정한다'는 것이 중요하다

3) pandas 의 sample(random_state=42) 도 같은 원리다

np.random.seed(): https://numpy.org/doc/stable/reference/random/generated/numpy.random.seed.html · Generator(권장): https://numpy.org/doc/stable/reference/random/generator.html

정수 배열 인덱싱과 정렬

정수 배열 인덱싱: 원하는 위치만 골라 꺼내기

```
sales = np.array([140, 180, 95, 230, 160])

sales[[0, 2, 4]]   # [140  95 160]

# 위치 목록을 대괄호 안에 넣는다
# 순서를 바꿔 꺼낼 수도 있다
sales[[4, 0]]      # [160 140]
```

정렬: np.sort 와 [::-1]

```
np.sort(sales)          # 오름차순
# [ 95 140 160 180 230]  작은 값부터

np.sort(sales)[::-1]    # 내림차순
# [230 180 160 140  95]  큰 값부터
# 원본 sales 는 그대로다
# [::-1] 은 "뒤에서부터" 슬라이싱

np.argmax(sales)  # 3  # 가장 큰 값의 위치
```

np.sort()는 기본적으로 오름차순 정렬한다. 정렬 결과를 [::-1]로 뒤집으면 내림차순 결과를 얻을 수 있다.

np.sort(): https://numpy.org/doc/stable/reference/generated/numpy.sort.html

불리언 인덱싱(boolean indexing)

대괄호에 위치 번호 대신 True/False 배열을 넣어, True 인 자리의 값만 골라내는 방법이다

sales	140	180	95	230	160
sales >= 150	False	True	False	True	True
			↓ True 자리만 남긴다		
sales[sales >= 150]	180	230	160		

```
sales = np.array([140, 180, 95, 230, 160])

mask = sales >= 150         # [False  True False  True  True]
sales[mask]                 # [180 230 160]

sales[sales >= 150]         # 한 줄로 쓰면 이렇게
(sales >= 150).sum()        # 3  → True 의 개수 = 조건에 맞는 건수
```

배열에 조건을 쓰면(sales >= 150) 자리마다 True/False 가 적힌 같은 길이의 배열이 나온다. 이것을 마스크(mask)라고 부른다
NumPy 가 마스크와 원본을 자리마다 맞춰 보고, True 인 자리의 값만 모아 새 배열 [180, 230, 160] 을 돌려준다

불리언 배열 인덱싱: https://numpy.org/doc/stable/user/basics.indexing.html#boolean-array-indexing

배열 메서드

sales = np.array([140, 180, 95, 230, 160]) 일 때

함수	뜻	결과	함수	뜻	결과
`sales.mean()`	평균	161.0	`sales.max()`	최댓값	230
`sales.sum()`	합계	805	`sales.argmax()`	최댓값의 위치(인덱스)	3
`sales.min()`	최솟값	95	`sales.argmin()`	최솟값의 위치(인덱스)	2

ndarray 배열 메서드(Calculation): https://numpy.org/doc/stable/reference/arrays.ndarray.html#calculation

np.nan 이란: None 과 다르다

np.nan: Not a Number. "숫자가 아니다"를 나타내는 실수(float)형 값이다. 데이터에서 빠진 값(결측값)을 표시하는 데 쓴다
None: 파이썬 자체의 "값이 없음"을 나타내는 객체다. 숫자가 아니라서 계산에 쓸 수 없다

구분	np.nan
자료형	float
배열에 넣으면	배열 전체가 float 가 된다
계산하면	결과가 nan 이 된다 (5 + np.nan → nan)
같은지 비교하면	nan == nan 도 False

```
np.array([1, 2, np.nan])   # [ 1.  2. nan]  float64
np.array([1, 2, None])     # [1 2 None]     object
5 + np.nan                 # nan
np.nan == np.nan           # False  → 그래서 np.isnan() 으로 확인한다
```

수치 배열의 결측값은 np.nan 으로 표시하고, 있는지 확인은 == 가 아니라 np.isnan() 으로 한다

np.nan: https://numpy.org/doc/stable/reference/constants.html#numpy.nan · None(한국어): https://docs.python.org/ko/3/library/constants.html#None

np.where(): 조건에 따라 값 정하기

np.where(조건, 참일 때 값, 거짓일 때 값)

조건

temps >= 80

`[False True False True]`

→

참이면

"경고"

거짓이면 "정상"

→

결과

[정상 경고 정상 경고]

같은 길이의 새 배열

```
temps = np.array([72, 88, 79, 91])   # 설비 온도 4개
status = np.where(temps >= 80, "경고", "정상")
```

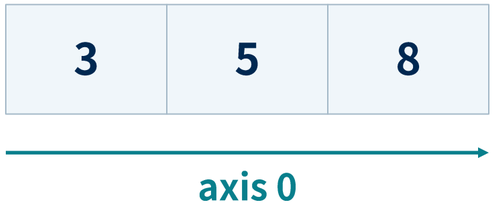
np.where: https://numpy.org/doc/stable/reference/generated/numpy.where.html

2차원 배열 · shape · axis · reshape · .T

배열의 차원

1차원 배열

벡터(Vector) = 1차원 배열, 값 한 줄



shape: (3,)

2차원 배열

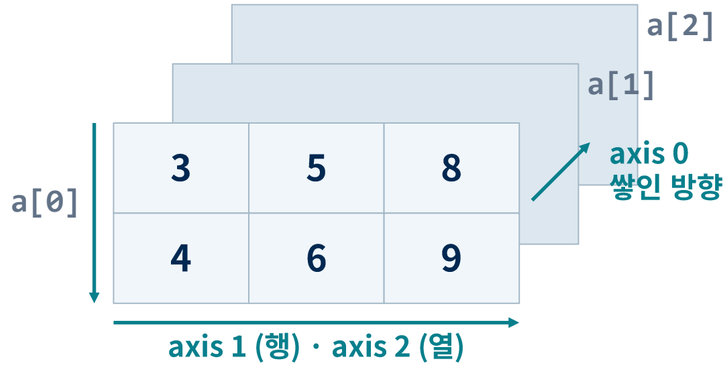
행렬(Matrix) = 2차원 배열, 값을 행과 열로 늘어놓은 것



shape: (2, 3)

3차원 배열

텐서(Tensor) = 3차원 이상 배열, 행렬을 쌓은 것



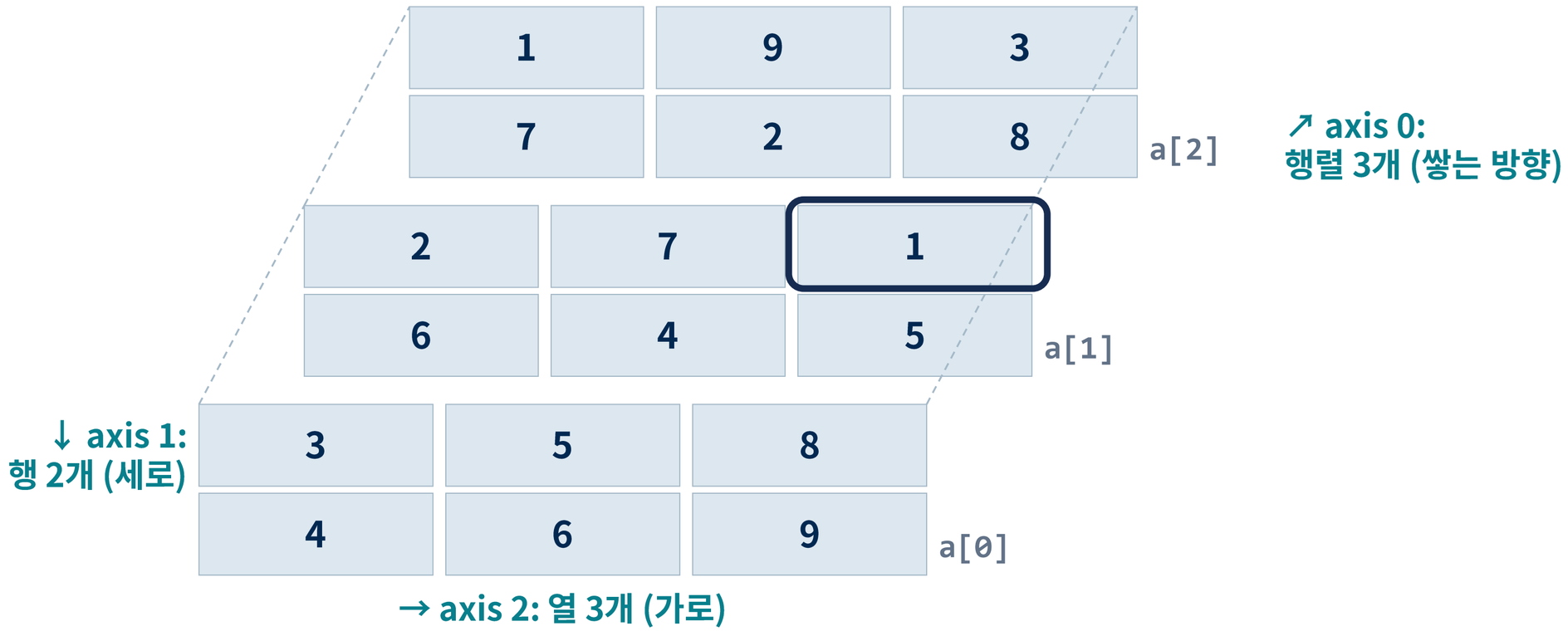
shape: (3, 2, 3)

axis(축): 배열의 차원을 부르는 다른 이름이다. 데이터가 늘어나는 방향으로 생각하면 쉽다.
번호는 바깥 대괄호부터 0, 1, 2 순서다. 2차원 배열에서 axis 0 은 세로(행), axis 1 은 가로(열)이다
ndim(차원): 축의 개수다. 1차원 = 벡터(값 한 줄), 2차원 = 행렬(행과 열), 3차원 = 행렬을 쌓은 것
shape(모양): 축마다 값이 몇 개인지를 (axis 0 의 크기, axis 1 의 크기, …) 순서로 적은 튜플이다. 그래서 len(shape) == ndim 이다
읽는 법: shape (3, 2, 3) = 행렬 3개 × 2행 × 3열.

NumPy 퀵스타트(3차원 배열, shape): https://numpy.org/doc/stable/user/quickstart.html · NumPy 용어집(axis): https://numpy.org/doc/stable/glossary.html

쌓인 모양으로 보는 3차원 배열

axis 0(층): 행렬 3개　axis 1(행): 행 2개　axis 2(열): 열 3개



a[1, 0, 2] = 1 : 두 번째 행렬, 첫 번째 행, 세 번째 열

axis 축지정

```
import numpy as np
a = np.array([[2, 4, 6],
              [8, 10, 12]])
print(a.shape)        # (2, 3)
print(a.sum(axis=0))  # [10 14 18]
print(a.sum(axis=1))  # [12 30]
print(a.sum())        # 42
```

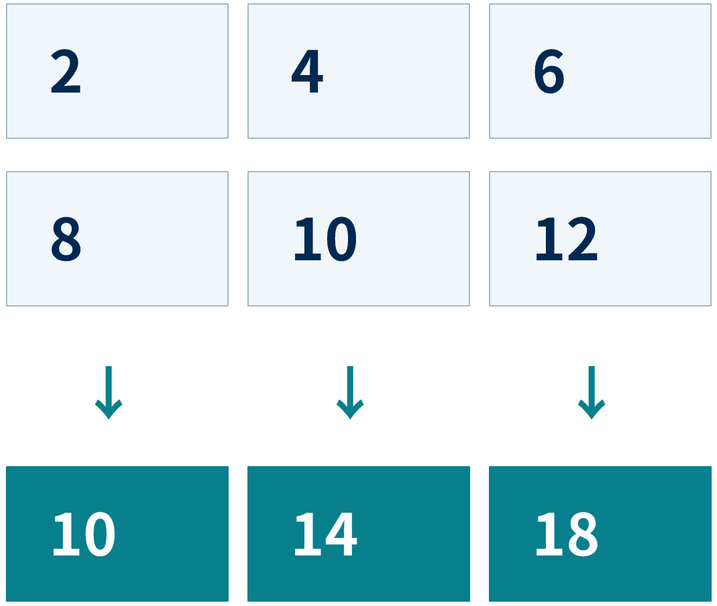
axis는 계산할 축(차원)을 지정하는 매개변수

지정	계산하는 방향	결과	shape 변화
axis=0	위→아래 (행끼리 합침)	열마다 하나: [10 14 18]	(2, 3) → (3,)
axis=1	왼쪽→오른쪽 (열끼리 합침)	행마다 하나: [12 30]	(2, 3) → (2,)
axis=None (기본값)	전체 원소	값 하나: 42	숫자 하나

np.sum(): https://numpy.org/doc/stable/reference/generated/numpy.sum.html

sum(axis)

1) axis=0 · 행을 다룬다: 위아래로 합치기



결과: 각 열의 합 [10, 14, 18]

shape (2, 3) → (3,) : 행 2개가 하나로 합쳐진다

2) axis=1 · 열을 다룬다: 좌우로 합치기



결과: 각 행의 합 [12, 30]

shape (2, 3) → (2,) : 열 3개가 하나로 합쳐진다

reshape

reshape: 배열의 값과 개수는 그대로 둔 채 모양(shape)만 바꾸는 함수다.
-1 은 전체 개수에 맞춰 그 자리 크기를 자동으로 계산하라는 뜻이다

reshape(행, 열)

```
np.arange(1, 7)  # [1 2 3 4 5 6]

np.arange(1, 7).reshape(2, 3)
# [[1 2 3]
#  [4 5 6]]
# 값은 그대로, 2행 3열로

np.arange(1, 7).reshape(4, 2)  # 오류
```

-1 은 '나머지에서 자동 추론'

```
values = np.arange(1, 13)  # 값 12개

values.reshape(3, -1)  # (3, 4)
# 열은 12 ÷ 3
```

코드	의미	결과
`values.reshape(-1, 1)`	열은 1개, 행은 자동 계산	(12, 1)
`values.reshape(1, -1)`	행은 1개, 열은 자동 계산	(1, 12)
`values.reshape(-1)`	원소 개수에 맞게 1차원으로 변환	(12,)

reshape(): https://numpy.org/doc/stable/reference/generated/numpy.reshape.html

reshape

값을 앞에서부터 차례로 꺼내, 한 행을 다 채우면 다음 행으로 넘어간다.

`np.arange(1, 7)` shape (6,)

1	2	3	4	5	6

↓ `.reshape(2, 3)`

1	2	3
4	5	6

1행: 1 2 3
2행: 4 5 6
shape (2, 3)

`np.arange(1, 13)` shape (12,)

1	2	3	4	5	6	7	8	9	10	11	12

↓ `.reshape(3, -1)`

1	2	3	4
5	6	7	8
9	10	11	12

행은 3 으로 지정
열은 -1: 전체 개수와 나머지 차원에서 자동 추론
→ 12 ÷ 3 = 4
shape (3, 4)

1	2
3	4
5	6
?	?

`np.arange(1, 7).reshape(4, 2)` → **오류**
칸은 4 × 2 = 8개인데 값은 6개뿐이라 빈칸이 생긴다. 행 × 열 = 값 개수 가 맞아야 한다

전치 .T 와 (2,) vs (2, 1)

전치(transpose): 행렬의 행과 열을 서로 맞바꾸는 연산이다.
.T 를 붙이면 되고, 값은 그대로이고 위치만 바뀐다

전치: 행과 열 바꾸기

```
production        # (3, 2)
[[86 92]          # 3회 측정 × 설비 2대
 [88 95]
 [84 90]]

production.T      # (2, 3)
[[86 88 84]       # 설비 2대 × 3회 측정
 [92 95 90]]
# .T[0] 은 M01 의 전체 값
```

1차원 배열은 .T 를 해도 그대로다

```
np.array([86, 92])
# (2,) 1차원

np.array([86, 92]).T
# (2,) 그대로

np.array([86, 92]).reshape(-1, 1)
# [[86]   (2, 1) 2행 1열
#  [92]]

np.array([86, 92]).reshape(1, -1)
# [[86 92]]   (1, 2) 1행 2열

np.array([86, 92]).reshape(1, -1).T  # [[86]
(2, 1) 2행 1열 : 2차원이라 전치가 된다#    [92]]
```

ndarray.T: https://numpy.org/doc/stable/reference/generated/numpy.ndarray.T.html

전치 .T

`production (3, 2)`

	M01	M02
1회	86	92
2회	88	95
3회	84	90

→

`.T (행 ↔ 열)`

`production.T (2, 3)`

	1회	2회	3회
M01	86	88	84
M02	92	95	90

.T[0] 은 첫 행
= M01 의 전체 값
[86 88 84]

1차원 (2,) : 행·열이 없다

86	92

.T →

86	92

모양이 그대로다

2차원 (2, 1) : 세로 한 열

86
92

.T →

86	92

(1, 2)
가로 한 행